

Word Vectors, Embeddings & Semantics in Vector Space

KAUST–Mawhiba IOAI Training

Week 3 · Day 13



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

1. Recap and Motivation
2. TF-IDF: Smarter Feature Extraction
3. Vector Space Models – Introduction
4. One-Hot vs. Dense Vectors
5. Word-by-Word vs. Word-by-Document Designs
6. Similarity Measures
7. Word2Vec
8. GloVe
9. fastText
10. Summary and Future Directions

- ▶ NLP enables machines to process human language
- ▶ **Tokenization**: splitting text into meaningful units
- ▶ **Vocabulary**: the set of all unique tokens
- ▶ **N-grams**: capturing local word co-occurrence
- ▶ **Bag of Words**: representing documents as count vectors

Problem with BoW:

- ▶ High-dimensional and sparse
- ▶ “happy” and “joyful” are completely unrelated
- ▶ Common words like “the” dominate

1. **TF-IDF**: smarter weighting for important words
2. **Vector Space Models**: meaning as geometry
3. **One-Hot vs. Dense Vectors**: sparse to dense
4. **Co-occurrence Matrices**: word-by-word, word-by-document
5. **Similarity Measures**: Euclidean distance, cosine similarity
6. **Word2Vec**: learning embeddings from context
7. **GloVe & fastText**: alternative embedding methods

Goal: From counting words → understanding meaning in vector space.

TF-IDF: Smarter Feature Extraction

Problem with raw term frequency:

- ▶ Words like “the”, “is”, “a” appear in *every* document
- ▶ They get the highest counts but carry the least information
- ▶ Rare, informative words (“neural”, “gradient”) get drowned out

Idea: Weight words by how *unique* they are across documents.

- ▶ A word that appears in *every* document → low weight
- ▶ A word that appears in *few* documents → high weight

How often does word t appear in document d ?

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total words in } d}$$

Example: Document = “I love NLP and I love AI”

- ▶ $\text{TF}(\text{“love”}, d) = \frac{2}{7} \approx 0.29$
- ▶ $\text{TF}(\text{“NLP”}, d) = \frac{1}{7} \approx 0.14$
- ▶ $\text{TF}(\text{“and”}, d) = \frac{1}{7} \approx 0.14$

How rare is word t across all documents?

$$\text{IDF}(t) = \log \frac{N}{|\{d : t \in d\}|}$$

where N = total number of documents.

Example: 1000 documents

- ▶ “the” appears in 1000 docs $\rightarrow \text{IDF} = \log \frac{1000}{1000} = 0$
- ▶ “neural” appears in 50 docs $\rightarrow \text{IDF} = \log \frac{1000}{50} = 3.0$
- ▶ “backpropagation” appears in 5 docs $\rightarrow \text{IDF} = \log \frac{1000}{5} = 5.3$

💡 Rare words get higher IDF scores!

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Worked Example:

Word	TF	IDF	TF-IDF
“the”	0.15	0.0	0.0
“neural”	0.05	3.0	0.15
“backprop”	0.02	5.3	0.106

Result: “the” is zeroed out. “neural” and “backprop” carry the signal.

Each document becomes a TF-IDF vector $\in \mathbb{R}^{|V|}$ — ready for similarity comparison, search, or classification.

Applications:

- ▶ **Search engines:** rank documents by relevance to a query
- ▶ **Text classification:** TF-IDF features → classifier
- ▶ **Document clustering:** group similar documents
- ▶ **Keyword extraction:** words with highest TF-IDF in a document

In Python (scikit-learn):

```
from sklearn.feature_extraction.text import TfidfVectorizer  
vectorizer = TfidfVectorizer()  
X = vectorizer.fit_transform(documents)
```

💡 TF-IDF bridges the gap between raw counts (BoW) and learned embeddings (Word2Vec).

Vector Space Model: **Introduction**

Why learn vector space models?

Where are you heading?

Where are you from?



Different meaning

What is your age?

How old are you?



Same Meaning

- ▶ Words and sentences can have different meanings depending on context.
- ▶ Vector space models help capture semantic similarity and differences.
- ▶ Useful for tasks like paraphrase detection, question answering, and information retrieval.

Firth, 1957

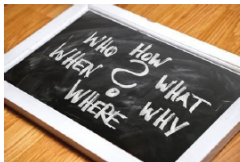


(Firth, J. R. 1957:11)

- ▶ VSM represents words or documents as vectors in an n -dimensional space.
- ▶ Based on the **distributional hypothesis**: Words that occur in similar contexts have similar meanings.
- ▶ Forms the foundation for information retrieval, document classification, and word embeddings.

- ▶ A **term-document matrix**: Rows represent words (terms), columns represent documents (or vice versa).
- ▶ Each cell contains a value such as:
 - Term Frequency (TF)
 - TF-IDF (Term Frequency-Inverse Document Frequency)
 - Co-occurrence count
- ▶ The matrix is typically high-dimensional and sparse:
 - $\text{Size} = |\text{Vocabulary}| \times |\text{Documents}|$

- ▶ You eat *cereal* from a *bowl* → Capturing semantic similarity (paraphrase understanding)
- ▶ You *buy* something and someone else *sells* it → Capturing relational meaning (analogies)



Information Extraction



Machine Translation



Chatbots

- ▶ Text is symbolic; machines require numerical representations to process it.
- ▶ Classical NLP struggled with sparse, high-dimensional vectors.
- ▶ Vector Space Models (VSMs) and Word Embeddings represent words in a continuous vector space.
- ▶ These models capture semantic meaning and relationships geometrically.
- ▶ Basis for modern NLP methods including Transformers.

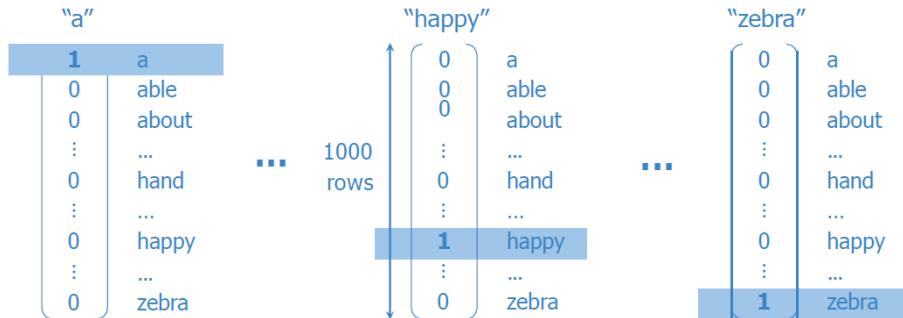
Word Embedding: **One-Hot vs. Dense Vectors**

Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

- + Simple
- Ordering: little semantic sense

hand < happy < zebra
615 621 1000
?! ?!

One-hot vectors



One-hot vectors (cont.)

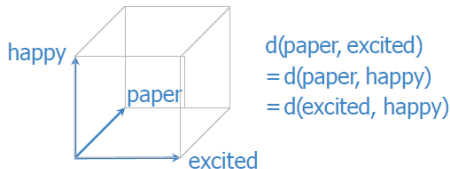
Word	Number			
a	1			
able	2			
about	3			
...	...			
hand	615			
...	...			
happy	621	←	621	1
...	...			
zebra	1000		1000	0

"happy"

1 0 a
2 0 able
3 0 about
... : ...
615 0 hand
... : ...
621 1 happy
... : ...
1000 0 zebra

- + Simple
- + No implied ordering
- Huge vectors
- No embedded meaning

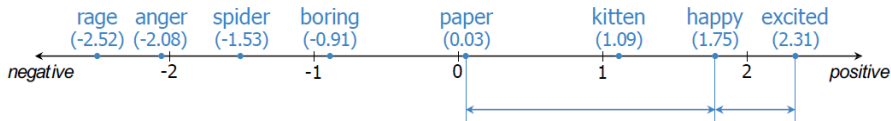
$$\begin{array}{c} \sim 10k-1M+ \\ \text{rows} \end{array} \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix} \begin{array}{l} a \\ \dots \\ \text{happy} \\ \dots \\ \text{Zyzyva} \end{array}$$



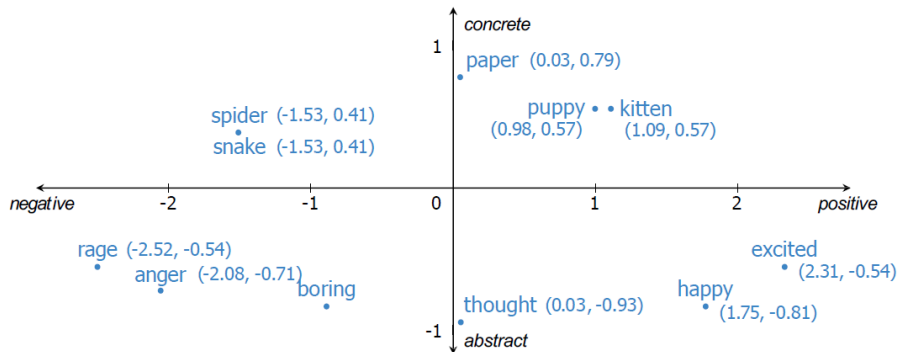
- ▶ Each word is represented as a unique vector.
- ▶ Vector has 1 at one position, 0 elsewhere.
- ▶ **Problems:**
 - High-dimensional and sparse.
 - No semantic meaning or similarity captured.

- ▶ Vectors are learned from data.
- ▶ Low-dimensional (e.g., 100–300).
- ▶ Capture semantic relationships.
- ▶ Example: "king" – "man" + "woman" $\approx$ "queen"

Meaning as vectors



Meaning as vectors (cont.)



+ Low dimension

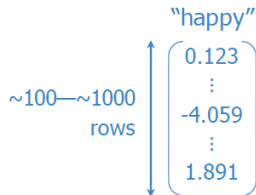
+ Embed meaning

- e.g. semantic distance

forest $\approx$ tree forest $\not\approx$ ticket

- e.g. analogies

Paris:France :: Rome:?



integers

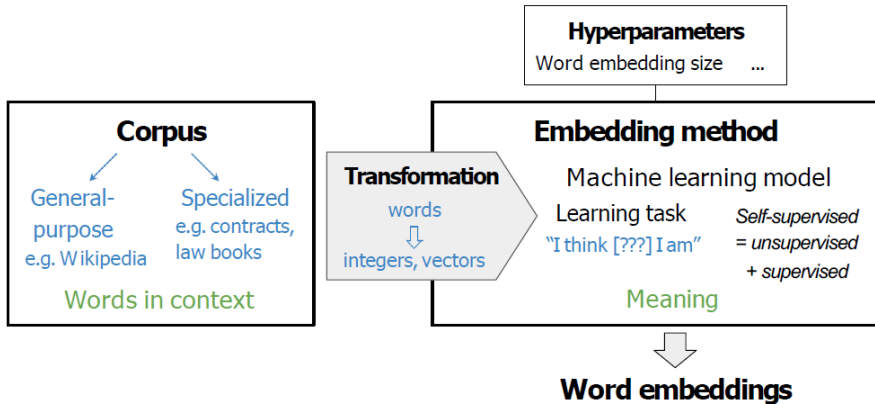
word vectors

one-hot vectors

word embedding vectors

"word vectors"

word embeddings



Word-by-Word vs Word-by-Document Designs

- ▶ Co-occurrence $\rightarrow$ Vector representation
- ▶ Relationships between words/documents

Number of times they *occur together within a certain distance k*

I like simple data
I prefer simple raw data

$k=2$

	simple	raw	like	I
data	2	1	1	0

n

Number of times a word *occurs within a certain category*

	Corpus		
	Entertainment	Economy	Machine Learning
data	500	6620	9320
film	7000	4000	1000

► Word-by-Word:

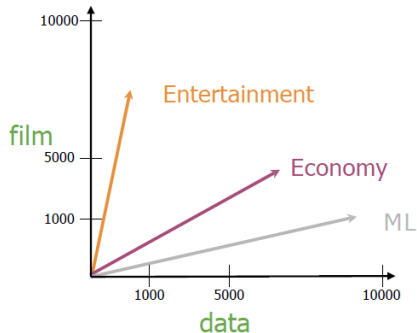
- Matrix built from co-occurrence counts between words.
- Captures semantic similarity directly.
- Better for word similarity tasks.

► Word-by-Document:

- Matrix built from word frequencies in documents.
- Useful for document classification and search.
- Better for document-level tasks.

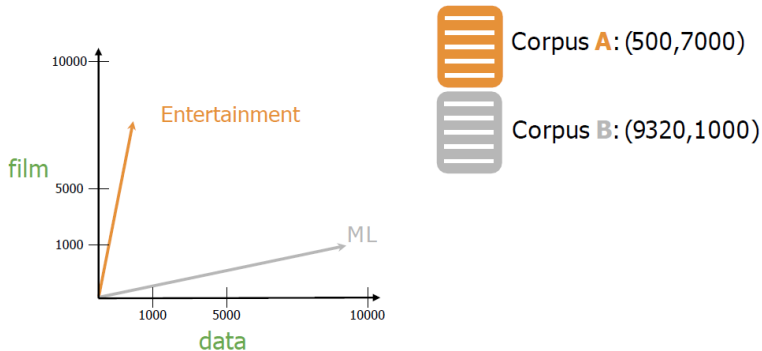
Tradeoffs: Choice depends on the task: word similarity vs. document analysis.

Similarity Measures



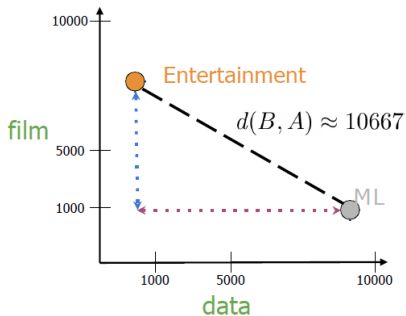
	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

Measures of "similarity:"
Angle
Distance



- Measures the straight-line distance between two points in space.
- Formula: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Sensitive to the scale (magnitude) of the vectors.

Euclidean Distance (cont.)



Corpus **A**: (500,7000)



Corpus **B**: (9320,1000)

$$d(B, A) = \sqrt{(B_1 - A_1)^2 + (B_2 - A_2)^2}$$
$$c^2 = a^2 + b^2$$

$$d(B, A) = \sqrt{(-8820)^2 + (6000)^2}$$

Euclidean Distance for n-dimensional vectors

	data	$\vec{w}$ boba	$\vec{v}$ ice-cream
AI	6	0	1
drinks	0	4	6
food	0	6	8

$$= \sqrt{(1 - 0)^2 + (6 - 4)^2 + (8 - 6)^2}$$

$$= \sqrt{1 + 4 + 4} = \sqrt{9} = 3$$

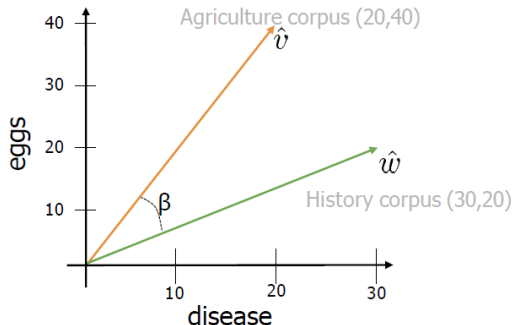
$$d(\vec{v}, \vec{w}) = \sqrt{\sum_{i=1}^n (v_i - w_i)^2} \longrightarrow \text{Norm of } (\vec{v} - \vec{w})$$


```
# Create numpy vectors v and w
v = np.array([1, 6, 8])
w = np.array([0, 4, 6])

# Calculate the Euclidean distance d
d = np.linalg.norm(v-w)

# Print the result
print("The Euclidean distance between v and w is: ", d)
```

The Euclidean distance between v and w is: 3



$$\hat{v} \cdot \hat{w} = \|\hat{v}\| \|\hat{w}\| \cos(\beta)$$

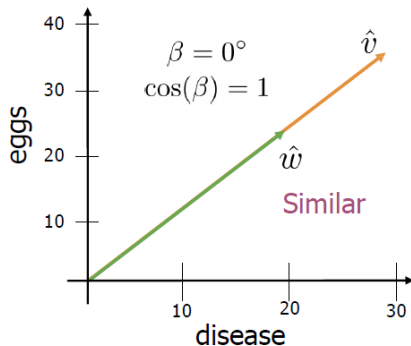
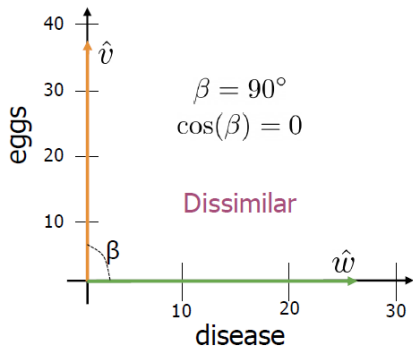
$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}} = 0.87$$

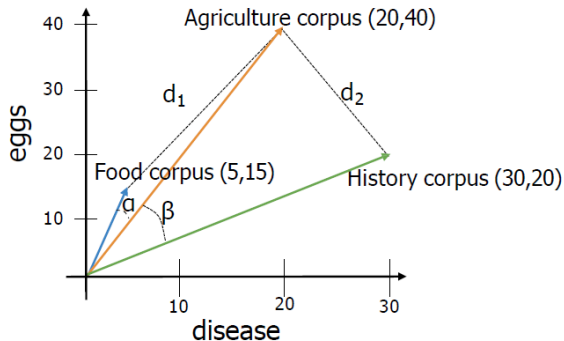
- ▶ Measures the cosine of the angle between two vectors.
- ▶ Formula: $\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$
- ▶ Ranges from -1 (opposite) to 1 (same direction).

- ▶ Less sensitive to magnitude, focuses on orientation.

Cosine Similarity (cont.)



- Cosine similarity when corpora are different sizes



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle between the vectors

Manipulating word vectors



USA



Washington
DC

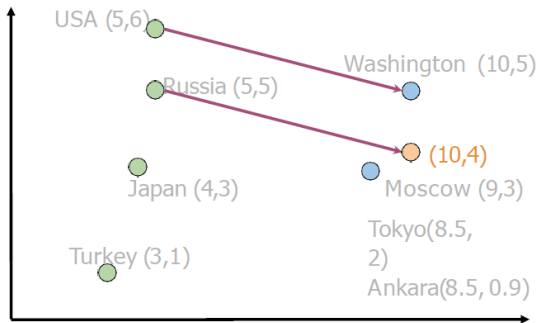


Russia



?

Manipulating word vectors (cont.)



$$\text{Washington} - \text{USA} = \begin{bmatrix} 5 & -1 \end{bmatrix}$$

$$\text{Russia} + \begin{bmatrix} 5 & -1 \end{bmatrix} = \begin{bmatrix} 10 & 4 \end{bmatrix}$$



Moscow

[Mikolov et al, 2013, Distributed Representations of Words and Phrases and their Compositionality]

	$d > 2$		
oil	0.20	...	0.10
gas	2.10	...	3.40
city	9.30	...	52.1
town	6.20	...	34.3

How can you visualize if your representation captures these relationships?



oil & gas

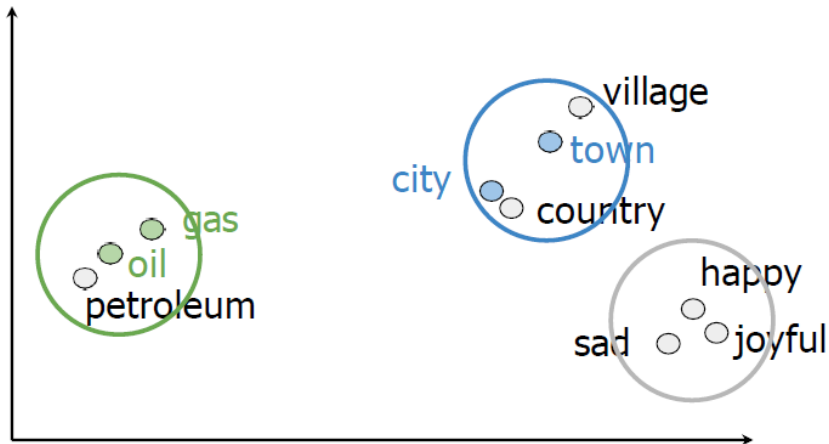


town & city

Visualization of word vectors (cont.)

$d > 2$							
oil	0.20	...	0.10	PCA	oil	2.30	21.2
gas	2.10	...	3.40		gas	1.56	19.3
city	9.30	...	52.1		city	13.4	34.1
town	6.20	...	34.3		town	15.6	29.8

Visualization of word vectors (cont.)



Embedding Method: **Word2Vec**

- ▶ Dense vector representations of words learned from context.
- ▶ Encode syntactic and semantic meaning.
- ▶ Vectors reflect relationships like synonymy, analogy, etc.

- ▶ **word2vec** (Google, 2013)
 - Continuous bag-of-words (CBOW)
 - Continuous skip-gram / Skip-gram with negative sampling (SGNS)
- ▶ **Global Vectors (GloVe)** (Stanford, 2014)
- ▶ **fastText** (Facebook, 2016)
 - Supports out-of-vocabulary (OOV) words

- ▶ Deep learning-based, contextual embeddings:
 - **BERT** (Google, 2018)
 - **ELMo** (Allen Institute for AI, 2018)
 - **GPT-2** (OpenAI, 2018)
- ▶ Tunable pre-trained models available

- ▶ Word2Vec learns word embeddings from large text corpora.
- ▶ Two main architectures:
 - Continuous Bag-of-Words (CBOW)
 - Skip-gram
- ▶ Both models use neural networks to predict words based on context.

- ▶ **Goal:** Predict target word from context.
- ▶ **Architecture:**
 - **Input:** Surrounding words (context)
 - **Output:** Target word
- ▶ Fast and accurate for frequent words.
- ▶ **Example:**
 - Context: “the ____ sat on the mat” → Predict “cat”

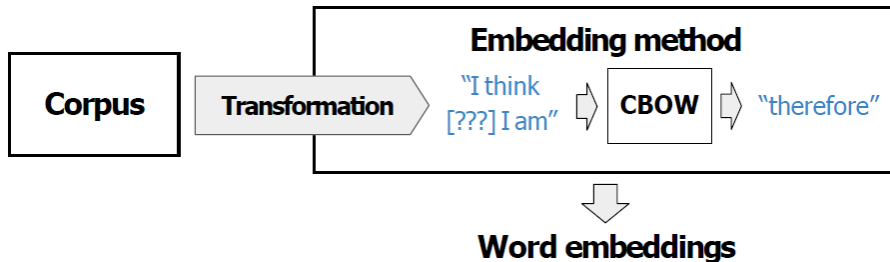


Figure 2: CBOW Architecture



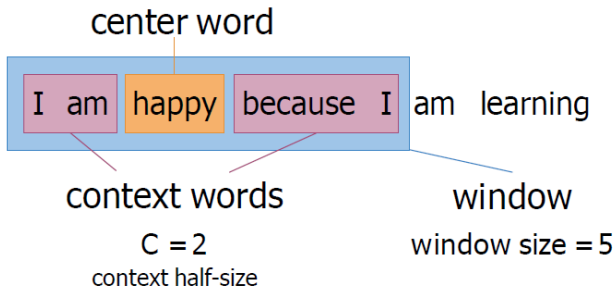
The little _____ ? _____ is barking

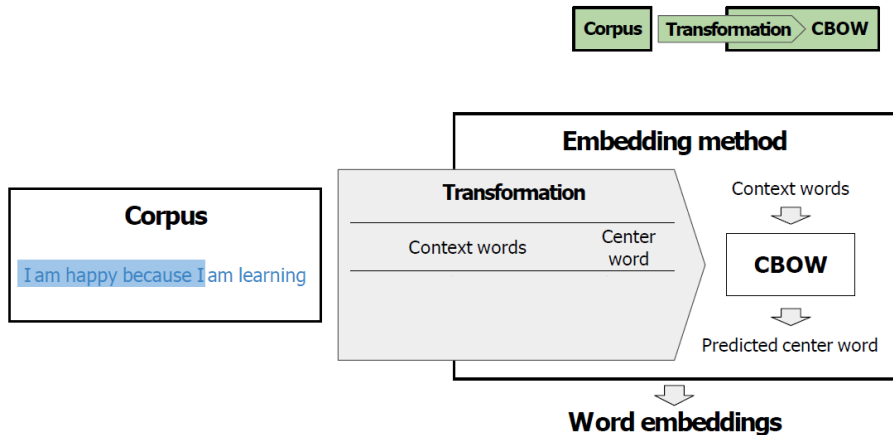


dog
puppy
hound
terrier

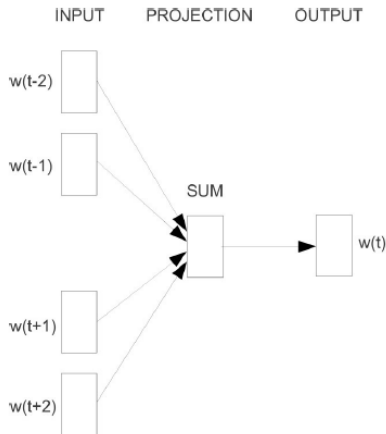
...

Creating a training example



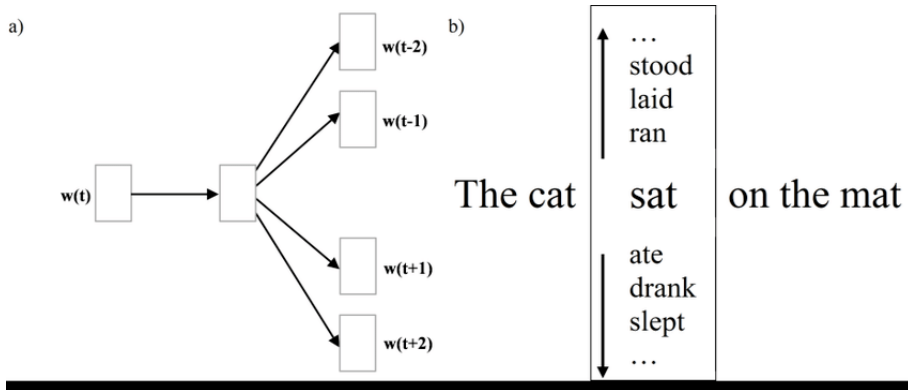


- ▶ **Goal:** Predict context words from a target word.
- ▶ **Architecture:**
 - **Input:** Target word
 - **Output:** Surrounding words (context)
- ▶ Better for rare words.
- ▶ **Example:**
 - Input: “cat” → Output: “the”, “sat”, “on”, “the”, “mat”



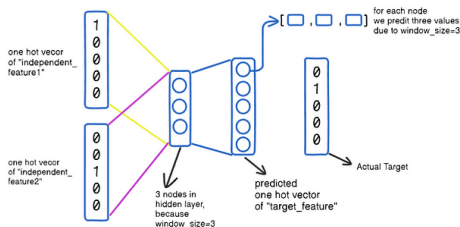
Source: Mikolov, T., Chen, K.,
Corrado, G.S., & Dean, J. (2013).
[Efficient Estimation of Word
Representations in Vector Space](#)

Figure 3: Skip-Gram Architecture

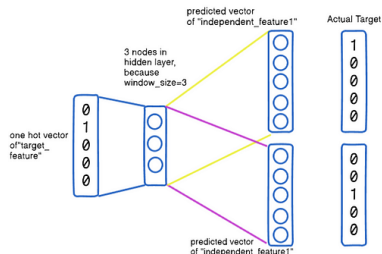


Word2vec

CBOW



Skip-gram



- ▶ Does not consider sub-word information.
- ▶ Same vector for all senses of a word (polysemy issue).
- ▶ Ignores word order within the context window.

GloVe – Global Vectors

Proposed by: Pennington et al., 2014

- ▶ Combines global matrix factorization with local context windows.
- ▶ Captures co-occurrence statistics of words across entire corpus.
- ▶ Embeddings reflect ratios of co-occurrence probabilities.

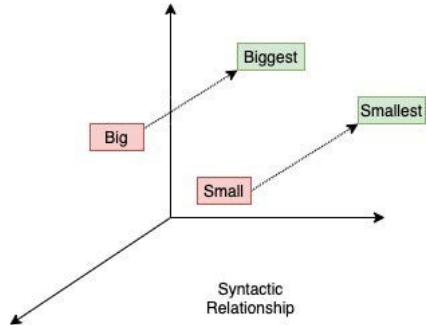
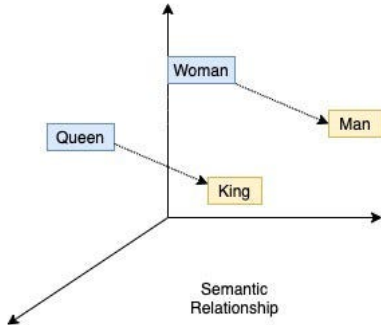
Loss function:

$$J = \sum_{i,j=1}^V f(P_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

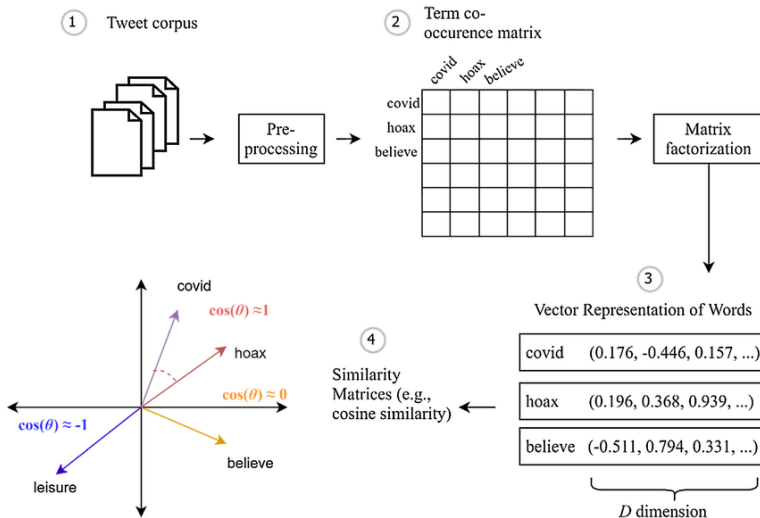
where:

- ▶ X_{ij} : number of times word j occurs in the context of word i
- ▶ P_{ij} : probability of word j in the context of i
- ▶ $w_i, \tilde{w}_j$: word and context word vectors
- ▶ $b_i, \tilde{b}_j$: bias terms
- ▶ f : weighting function

GloVe – Global Vectors (cont.)



GloVe – Global Vectors (cont.)



fastText – Subword Embeddings

- ▶ Developed by Facebook AI (2016)
- ▶ Builds on Word2Vec by incorporating character n-grams
- ▶ Useful for morphologically rich languages and rare words
- ▶ Better handling of OOV (out-of-vocabulary) words

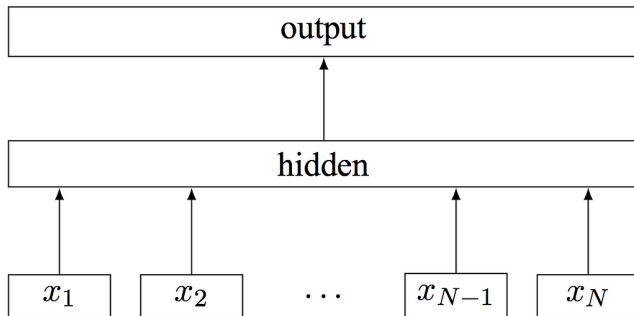
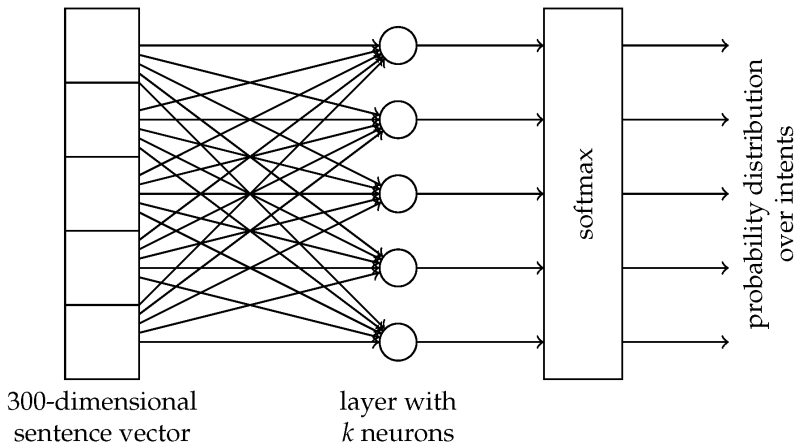


Figure 1: Model architecture of `fastText` for a sentence with N ngram features $x_1, \dots, x_N$. The features are embedded and averaged to form the hidden variable.



FastText-Based Intent Detection for Inflected Languages

- ▶ Word: playing
- ▶ Character n-grams (n=3): ["pla", "lay", "ayi", "yin", "ing"]
- ▶ Word vector = sum of n-gram vectors

Comparison: Word2Vec vs. GloVe vs. fastText

Feature	Word2Vec	GloVe	fastText
Local Context	✓	—	✓
Global Info	—	✓	✓ (partial)
Subword Info	—	—	✓
Handles OOV	—	—	✓

Summary

- ▶ **Contextual embeddings** (ELMo, BERT, GPT): Word vectors depend on sentence context.
- ▶ **Multilingual embeddings** and cross-lingual models.
- ▶ **Graph-based embeddings** (e.g., knowledge graph completion).
- ▶ **Hybrid embeddings**: Combining structured and unstructured data.

- ▶ Vector Space Models (VSMs) are foundational for modern NLP.
- ▶ Word embeddings capture semantic relationships in a continuous space.
- ▶ Word2Vec, GloVe, and fastText are key methods for generating word vectors.
- ▶ Dense embeddings outperform one-hot vectors in capturing meaning and relationships.
- ▶ Future work focuses on contextual, multilingual, and hybrid embeddings.

Credits

KAUST Academy

King Abdullah University of Science and Technology

Vector space and word embedding content adapted from
KAUST Academy NLP course materials

KAUST–Mawhiba IOAI 2026 Training Program